

0624-下午：数据库表结构设计

一、知识点

- 关系型数据库三大范式（1NF / 2NF / 3NF）
- ER 模型设计（实体、属性、关系）
- 主键策略（自增 / UUID / 雪花算法）
- 索引设计（主键索引 / 唯一索引 / 普通索引 / 联合索引）
- AI 伴学平台数据库完整设计

二、内容

1、为什么要学表结构设计

上午学了"怎么查", 现在要学"怎么建"。表设计得好, 查询简单、性能高; 设计得差, 到处要 JOIN、查询慢、改需求要改表。

前端超时时间: 10s, 程序和数据库执行7s 100ms

比如:

差的设计: 一个 `user` 表存了 50 个字段, 包含了地址、订单、学习记录.....
好的设计: 拆成 `user`、`user_address`、`order`、`learning_record` 多张表, 各管各的

2、三大范式

2.1 第一范式 (1NF) : 字段不可再分

规则: 每个字段必须是原子值, 不能再拆分。

```
-- ❌ 违反 1NF: 一个字段存了多个值
CREATE TABLE user (
  id          BIGINT PRIMARY KEY,
  skills      VARCHAR(200)  -- "JavaSE,JavaWeb,JavaEE-XX,MySQL,Vue"  ← 一个字段存多个技能, 无法
  查询
);

-- ✅ 符合 1NF: 拆成独立的关联表
CREATE TABLE user_skill (
  user_id    BIGINT,
  skill_id   BIGINT,
  PRIMARY KEY (user_id, skill_id)
);
```

2.2 第二范式 (2NF)：非主键字段完全依赖于主键

规则：在满足 1NF 的基础上，非主键字段必须完全依赖于整个主键（不能只依赖主键的一部分）。

```
-- ✗ 违反 2NF: skill_name 只依赖 skill_id, 不依赖 user_id
CREATE TABLE user_skill_record (
  user_id    BIGINT,
  skill_id   BIGINT,
  skill_name VARCHAR(100),  -- ← 冗余! 只和 skill_id 有关
  study_time INT,
  PRIMARY KEY (user_id, skill_id)
);

-- ✓ 符合 2NF: 把 skill_name 拆到 skill 表
CREATE TABLE user_skill_record (
  user_id    BIGINT,
  skill_id   BIGINT,
  study_time INT,
  PRIMARY KEY (user_id, skill_id)
);
-- skill_name 在 skill 表中, 通过 JOIN 查
```

2.3 第三范式 (3NF)：非主键字段不能传递依赖

规则：在满足 2NF 的基础上，非主键字段不能依赖于其他非主键字段。

```
-- ✗ 违反 3NF: department_name 依赖 department_id, department_id 依赖 id
CREATE TABLE employee (
  id            BIGINT PRIMARY KEY,
  name          VARCHAR(50),
  department_id BIGINT,
  department_name VARCHAR(100)  -- ← 传递依赖: name → department_id → department_name
);

-- ✓ 符合 3NF: department_name 存在 department 表中
CREATE TABLE employee (
  id            BIGINT PRIMARY KEY,
  name          VARCHAR(50),
  department_id BIGINT
);

-- department_name 通过 JOIN 查

A - B - C - D  Sql
```

2.4 范式总结

范式	核心要求	通俗理解
1NF	字段原子性	一个格子只存一个值
2NF	完全依赖主键	不能有"半依赖"的字段

范式	核心要求	通俗理解
3NF	不能传递依赖	$A \rightarrow B \rightarrow C$, C 不应该出现在 A 的表里

实际项目中：不必死守范式，有时为了查询性能会故意冗余（反范式化）。但冗余要有理由，比如"这个查询频率极高、JOIN 性能差"。

3、ER 模型设计

3.1 核心概念

概念	含义	示例
实体 (Entity)	业务中的"对象"	用户、技能、学习记录
属性 (Attribute)	实体的特征	用户的用户名、邮箱
关系 (Relationship)	实体之间的关联	用户"学习"技能

3.2 关系类型

一对一 (1:1) 用户 ↔ 用户详情

一对多 (1:N) 用户 ↔ 学习记录

多对多 (M:N) 用户 ↔ 技能(skillId) (通过中间表)

后台设计技能树，用户可以选择，学习点亮树里面的节点

user、skill 、 user_skill

主表user t1, 关联user_skill t2 on t1 = t2 , skill t3 on t2 = t3

user --> user_skill --> skill

一对多： 在"多"的一方加外键

```
-- 一个用户有多条学习记录
-- 外键加在 learning_record 表上
CREATE TABLE learning_record (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL COMMENT '关联用户ID',
  skill_id    BIGINT NOT NULL COMMENT '关联技能ID',
  -- ...
  INDEX idx_user_id (user_id)  -- 外键字段加索引
);
```

多对多： 用中间表

```
-- 一个用户可以学习多个技能，一个技能可以被多个用户学习
-- 需要中间表 user_skill
CREATE TABLE user_skill (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id     BIGINT NOT NULL,
    skill_id    BIGINT NOT NULL,
    level       TINYINT DEFAULT 1 COMMENT '掌握等级',
    UNIQUE KEY uk_user_skill (user_id, skill_id) -- 联合唯一索引，防止重复
);
```

4、主键策略

4.1 自增主键（推荐入门使用）

```
id BIGINT AUTO_INCREMENT PRIMARY KEY
```

- 优点：简单、有序、B+树索引友好
- 缺点：分库分表时会冲突、暴露数据量

4.2 UUID

```
// Java 中生成 UUID 设备码、时间。
String id = UUID.randomUUID().toString();
// 结果：550e8400-e29b-41d4-a716-446655440000
```

- 优点：全局唯一、不暴露数据量
- 缺点：无序、索引性能差、占空间大（36字符）

4.3 雪花算法（分布式场景推荐）、美团、百度 第三方库（订单号算法）

雪花ID（64位）= 1位符号 + 41位时间戳 + 10位机器ID + 12位序列号

- 优点：全局唯一、有序递增、高性能
- 缺点：需要引入依赖

5、索引设计

5.1 什么是索引（优化）

索引 = 书的目录。没有索引，查数据要“全表扫描”（逐行查找）；有索引，直接定位到目标行。

没有索引：SELECT * FROM user WHERE username = 'zhangsan'
→ 逐行扫描 10000 行 → 找到 1 行

有索引：在 username 上建了索引
→ 通过 B+ 树直接定位 → 找到 1 行（毫秒级）

5.2 索引类型

```
-- 1. 主键索引（自动创建）
-- PRIMARY KEY (id)

-- 2. 唯一索引：保证字段值不重复
CREATE UNIQUE INDEX uk_username ON user(username);

-- 3. 普通索引：加速查询，允许重复
CREATE INDEX idx_user_id ON learning_record(user_id);

-- 4. 联合索引：多个字段组合（最左前缀原则） 多个条件
CREATE INDEX idx_user_skill ON learning_record(user_id, skill_id);
-- 可以加速：WHERE user_id = 1
-- 可以加速：WHERE user_id = 1 AND skill_id = 2
-- 不能加速：WHERE skill_id = 2（跳过了最左边的 user_id） // 索引失效 prefix
```

5.3 索引设计原则

select * from user_skill where userId = 1

原则	说明
外键必须加索引	user_id、skill_id 等关联字段
WHERE 条件字段加索引	频繁出现在 WHERE 中的字段
ORDER BY 字段加索引	排序字段有索引可以避免 filesort
不要过度索引	每个索引都会降低写入性能
联合索引注意顺序	高频查询字段放前面

5.4 查看和分析索引

```
-- 查看表的所有索引
SHOW INDEX FROM user;

-- EXPLAIN 分析查询是否使用了索引
EXPLAIN SELECT * FROM user WHERE username = 'zhangsan';

-- 关注的字段：
-- type: ALL（全表扫描）→ 差 | ref/range（使用索引）→ 好
-- key: 实际使用的索引名
-- rows: 预估扫描行数
```

6、AI 伴学平台数据库设计实战

根据项目需求，设计完整表结构：

6.1 用户模块

RBAC: user、role、user_role、meau、role_meau;

```
-- 用户表
CREATE TABLE sys_user (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  username    VARCHAR(50) NOT NULL UNIQUE COMMENT '用户名',
  password    VARCHAR(100) NOT NULL COMMENT '密码',
  nickname    VARCHAR(50) DEFAULT '' COMMENT '昵称',
  email       VARCHAR(100) DEFAULT '' COMMENT '邮箱',
  avatar      VARCHAR(255) DEFAULT '' COMMENT '头像',
  role        VARCHAR(20) DEFAULT 'student' COMMENT '角色: student/admin',
  status      TINYINT      DEFAULT 1 COMMENT '0-禁用 1-正常',
  deleted     TINYINT      DEFAULT 0 COMMENT '0-正常 1-已删除',
  create_time DATETIME     DEFAULT CURRENT_TIMESTAMP,
  update_time DATETIME     DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_status (status),
  INDEX idx_role (role)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='系统用户表';
```

6.2 技能树模块

```
1:Java    pid:0
2 JavaSE \ 3 JavaWeb \ 4 Spring \ SpringCloud \ OS    pid : 1
面向对象、字符串、集合、异常 pid:2 \ servlet、tomcat、cookies、session、mysql \ Spring。。 \
```

```
-- 技能表
CREATE TABLE skill (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL COMMENT '技能名称',
  category    VARCHAR(50) NOT NULL COMMENT '技能类别',
  description TEXT          DEFAULT NULL COMMENT '技能描述',
  level       TINYINT      DEFAULT 1 COMMENT '难度等级 1-5',
  parent_id   BIGINT       DEFAULT 0 COMMENT '父技能ID,0顶层id',
  sort_order  INT          DEFAULT 0 COMMENT '排序',
  create_time DATETIME     DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_category (category),
  INDEX idx_parent_id (parent_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='技能树表';

-- 用户技能关联表 ? 没数据的? 什么时候数据进来
-- 刚开始使用: 10颗技能树、每颗10个节点: 100个
-- 新用户登录进来: id和所有的skillid做关联?、字符串: 先判断关联表有没有数据, 没有的新增, 有就修改

-- 用户查询技能树, 树节点都可以查询到, 但是要做left join
-- userId: 1, Java (那个节点是点亮了, 是有数据的, null)
CREATE TABLE user_skill (
```

```

id          BIGINT AUTO_INCREMENT PRIMARY KEY,
user_id     BIGINT NOT NULL,
skill_id    BIGINT NOT NULL,
level       TINYINT DEFAULT 0 COMMENT '掌握程度 0-5',
status      TINYINT DEFAULT 0 COMMENT '0-未开始 1-学习中 2-已掌握',
update_time DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE KEY uk_user_skill (user_id, skill_id),
INDEX idx_user_id (user_id),
INDEX idx_skill_id (skill_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户技能关联表';

```

6.3 学习记录（技能点点亮）模块

user、order、address；选择收获地址

order (user_id\address_id) 实时查。历史数据关联最新外键信息

order_info（下单时、多少钱、截图）快照

```

-- 学习记录表
CREATE TABLE learning_record (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  skill_id    BIGINT NOT NULL COMMENT '学习资源ID',
  skill_name  VARCHAR(200) DEFAULT '' COMMENT '资源名称（冗余，减少JOIN；历史数据快照）',
  progress    INT DEFAULT 0 COMMENT '进度百分比 0-100',
  study_seconds INT DEFAULT 0 COMMENT '累计学习秒数',
  status      TINYINT DEFAULT 0 COMMENT '0-未开始 1-学习中 2-已完成',
  first_study_time DATETIME DEFAULT NULL COMMENT '首次学习时间',
  last_study_time DATETIME DEFAULT NULL COMMENT '最近学习时间',
  complete_time DATETIME DEFAULT NULL COMMENT '完成时间',
  create_time  DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_id (user_id),
  INDEX idx_resource_id (resource_id),
  INDEX idx_user_status (user_id, status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='学习记录表';

```

反范式化设计： learning_record 中的 resource_name 是冗余字段（应该通过 JOIN resource 表获取）。但学习记录查询频率极高，冗余一个 name 可以避免每次查询都 JOIN，这是常见的性能优化手段。

6.4 AI 服务模块

AI功能：技能树点亮 -- AI出题（6）【速率、后台设计技能树可以先生成30道题目】 -- 【题目、作答信息】 -- AI判题（打分、回答建议、学习建议） -- 统分（60）

-- AI判题评分大于60 -- 修改当前用户关联（user_skill）、点亮

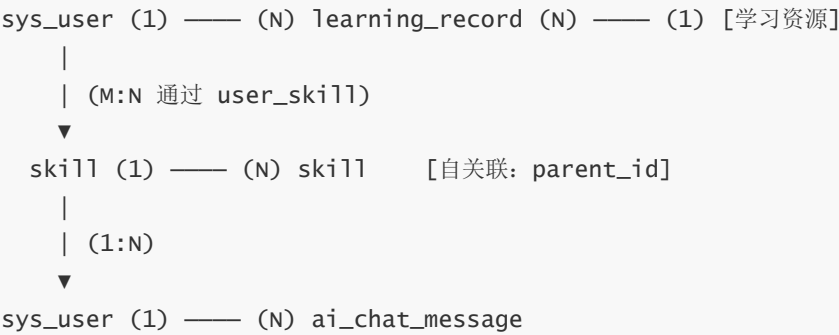
```

-- AI任务记录表

```

```
-- AI 对话记录表 Dify ()
CREATE TABLE ai_chat_message (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT      NOT NULL,
  session_id  VARCHAR(64) NOT NULL COMMENT '对话会话ID',
  role        VARCHAR(20) NOT NULL COMMENT 'user/assistant/system',
  content     TEXT        NOT NULL COMMENT '消息内容',
  model       VARCHAR(50) DEFAULT '' COMMENT '使用的模型',
  tokens_used INT         DEFAULT 0 COMMENT '消耗Token数',
  create_time DATETIME    DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_session (user_id, session_id),
  INDEX idx_create_time (create_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='AI对话记录表';
```

6.5 ER 图 (文字版)



7、课堂练习

- 1. 根据以上设计，在 MySQL 中执行建表语句，创建所有表
- 2. 为每张表插入 5-10 条测试数据
- 3. 使用 DESC 表名 和 SHOW INDEX FROM 表名 验证表结构和索引
- 4. 编写一个查询：查找某个用户"学习中"的所有技能名称和进度
- 5. 编写一个查询：统计每个技能类别下有多少用户在学习
- 6. 借助Agent设计一份完整项目数据库（贴合自己项目的功能）

三、课堂小结

主题	关键点
三大范式	1NF字段原子 → 2NF完全依赖 → 3NF无传递依赖
一对多	在"多"的一方加外键
多对多	用中间表，加联合唯一索引
主键策略	实训阶段用自增 BIGINT，微服务阶段用雪花算法

主题	关键点
索引	外键必加、WHERE/ORDER BY 高频字段加、联合索引注意最左前缀
反范式化	高频查询字段可适当冗余，用空间换时间